

Propuestas de modificaciones del proyecto 1

Tiempo estimado máximo: 15 minutos

Todas las modificaciones se deben verificar con un archivo de entrada adecuado para cada modificación.

1. Intercambiar los operadores adición por sustracción, multiplicación por división, AND por OR.

Entrada:

```
var a int32 = 3
var b int32 = 2
var b1 bool = true
var b2 bool = false
fmt.Println(a + b)
fmt.Println(a - b)
fmt.Println(a * b)
fmt.Println(a / b)
fmt.Println(b1 && b2)
fmt.Println(b1 || b2)
```

Salida:

```
1
5
1
6
true
false
```

2. Intercambiar el orden de precedencia de la adición con el de la multiplicación, el de la sustracción con el de la división, y el de AND con el de OR.

Entrada:

```
var a int32 = 3
var b int32 = 2
var b1 bool = true
var b2 bool = false
fmt.Println(a + b * b)
fmt.Println(a - b / b)
fmt.Println(b1 && b2 || b2)
```

Salida:

```
10
0
false
```

3. Implementar bucle do-while (ejecuta al menos una vez):

Entrada:

```

// Falsa desde el inicio
var x int32 = 10
for {
    fmt.Println(x)
    if !(x < 5) {
        break
    }
}
fmt.Println("Segundo while")
// Comportamiento normal
var i int32 = 0
for {
    fmt.Println(i)
    i++
    if !(i < 3) {
        break
    }
}

```

Salida:

```

10
Segundo while
0
1
2

```

4. Implementar operador de potencia (**) en las expresiones

Entrada:

```

var result int32 = 2 ** 3
fmt.Println(result)

```

Salida:

```

8

```

5. Switch con rangos para enteros (case x..y) Permite rangos de enteros en casos de switch donde ambos límites están incluidos, es decir es verdadero si variable >= x && variable <= y.

Entrada:

```

var score int32 = 85
switch score {
case 90..100:
    fmt.Println("A")
    break
case 80..89:
    fmt.Println("B")
    break
case 70..79:
    fmt.Println("C")
}

```

```

        break
    default:
        fmt.Println("F")
}

```

Salida:
B

6. Modificar arreglos de una dimensión para que el índice sea 1-based en lugar de 0-based.

Entrada:

```

var numeros [3]int32 = [3]int32{10, 20, 30}
fmt.Println(numeros[2])

```

Salida:
20

7. Modificar while para que ejecute el bloque una vez más luego de que la condición sea falsa.

Entrada:

```

var i int32 = 0
for i <= 2 {
    fmt.Println(i)
    i++
}

```

Salida:
0
1
2
3

8. Modificar la sentencia if para ejecutar las sentencias verdaderas cuando el valor de la expresión sea falsa.

Entrada:

```

if false {
    fmt.Println("Hola mundo")
}

```

Salida:
Hola mundo

9. Un case puede listar varias constantes separadas por |. Coincide si la variable es igual a cualquiera de ellas.

Entrada:

```

var d int32 = 6
switch d {
case 1|3|5|7|9:
    fmt.Println("Impar")
}

```

```

        break
    case 2|4|6|8|10:
        fmt.Println("Par")
        break
    default:
        fmt.Println("Otro")
}

```

Salida:
Par

10. Se agrega `^^` para XOR lógico (verdadero si exactamente uno es verdadero).

Entrada:

```

var a bool = true
var b bool = false
var c bool = true
fmt.Println(a ^^ b)
fmt.Println(a ^^ c)
fmt.Println(b ^^ c)

```

Salida:
true
false
true

11. Creación de una nueva estructura de repetición que ejecuta el bloque exactamente `n` veces.

Entrada:

```

repeat 3 {
    fmt.Println("Hola mundo")
}

```

Salida:
Hola mundo
Hola mundo
Hola mundo

12. En las funciones de tipo `void` la instrucción `return` debe ser ignorada.

Entrada:

```

func saludo() {
    return
    fmt.Println("Hola mundo")
}

func main() {
    saludo()
}

```

Salida:

Hola mundo

13. Permitir índices negativos en arreglos, arreglo[-1] tendría que acceder al último elemento y un arreglo[-2] tendría que acceder al penúltimo y así sucesivamente.

Entrada:

```
var nums [4]int32 = [4]int32{10, 20, 30, 40}
fmt.Println(nums[-1])
fmt.Println(nums[-3])
fmt.Println(nums[0])
```

Salida:

```
40
20
10
```

14. Bucle for con rango inclusivo -> for (i in x..y))

Entrada:

```
for i in 1..4 {
    fmt.Println(i)
}
```

Salida:

```
1
2
3
4
```

15. Un case puede listar varias constantes por medio de rango E1..E2. Coincide si la variable está dentro de algún rango.

Entrada:

```
var score int32 = 85
switch score {
case 90..100:
    fmt.Println("A")
    break
case 80..89:
    fmt.Println("B")
    break
case 70..79:
    fmt.Println("C")
    break
default:
    fmt.Println("F")
}
```

Salida:

```
B
```

16. Operador de comparación por rango: in [E1..E2] y not in [E1..E2]

Entrada:

```
var x int32 = 10
```

```
fmt.Println( x in [5..10] )
```

```
fmt.Println( x not in [11..20] )
```

Salida:

```
true
```

```
true
```